

Mastering Software Testing & Mocking in Spring Boot

A Deep Architectural Guide to JUnit 5, Mockito, and Spring Test Contexts

■ ARCHITECTURAL COURSE SYLLABUS:

This master handbook covers the entire testing lifecycle for Java microservices. It breaks down unit testing theory, the mechanical differences between mocks and stubs, provides absolute annotation definitions, walks through your Notification and Reporting test suites, and answers crucial system design concepts.

Chapter 1: The Philosophy of Software Testing

Software testing is the process of verifying that an application behaves exactly as expected. In professional development, we use **Automated Testing** instead of manual clicking to prevent regression bugs (bugs introduced when modifying existing code).

The Testing Pyramid:

1. **Unit Tests (Base):** Tests a single class/method in complete isolation. Highly mocked, runs in milliseconds. Your project's `NotificationServiceTest` is a unit test.
2. **Integration Tests (Middle):** Tests the interaction between multiple classes or database queries. Slower, uses real databases or containers.
3. **End-to-End (E2E) Tests (Top):** Tests the entire user flow from the frontend React UI all the way to backend services.

Chapter 2: The Art of Mocking vs. Stubbing

In unit testing, we must isolate our service class from external dependencies like database connections or Feign clients.

- **Mocking:** Creating a simulated, blank-canvas instance of a class. A mock object mimics the class signature but contains zero internal state and returns default values (null, false, or 0) for all methods by default.
- **Stubbing:** Programming the mock object to return a pre-configured response when called. Stubbing gives the mock its behavior. We use Mockito's `when().thenReturn()` syntax to register these behaviors.

Chapter 3: Absolute Dictionary of Test Annotations

Annotation	Exact Architectural Purpose
@ExtendWith(MockitoExtension.class)	Enables Mockito integration in JUnit 5. It manages mock lifecycles and automatically disposes of mocks after each test to prevent memory leaks.
@Mock	Tells Mockito to build a bytecode-generated dummy stub of the class (e.g. UserClient). Real code execution is bypassed.
@InjectMocks	Instantiates the real service under test (e.g. NotificationServiceImpl) and automatically injects all variables annotated with @Mock into its constructor.
@BeforeEach	Indicates that the annotated method must run before each individual @Test method . Used to set up fresh mock inputs and data templates.
@Test	Identifies a method as a executable test case. JUnit executes these methods and verifies assertions.

Chapter 4: Understanding the Spring Test Context & Context Loading

What is the Spring Context?

The Spring Context is the engine's internal registry where all Spring Beans (Services, JpaRepositories, DataSource pools) reside. When a Spring Boot application starts up, it constructs this Context container by reading configurations and connecting dependencies.

What is contextLoads() & @SpringBootTest?

The default test class created by Spring Initializr includes a contextLoads() test annotated with @SpringBootTest. Its only job is to boot up this entire warehouse registry to verify that there are no configuration or injection errors.

Why did we comment it out?

Because @SpringBootTest tries to establish real MySQL database connections and bind ports. In a lightweight unit test environment, we do not have our databases running. Booting up the entire context is expensive (takes 10-20 seconds) and fails without database connections. By commenting it out and using Mockito directly, our tests compile and run in a fraction of a second, with zero database dependencies!

Chapter 5: Detailed Mechanics of Your Code (How We Test it)

1. Stubbing standard database methods:

Since our service uses saveAndFlush() to commit transactions, we stubbed exactly that method:

```
when(notificationRepository.saveAndFlush(any(Notification.class))).thenReturn(mockNotification);
```

2. Simulating Downstream Service Crashes:

We tested our microservice fault tolerance by stubbing `UserClient` to throw an exception, and verified that the service catches it cleanly:

```
when(userClient.getUserById(1L)).thenThrow(new RuntimeException("Offline"));
```

3. Asserting Access Control & Exceptions:

We verified that a Tourist role gets rejected with an HTTP 403 Forbidden exception when trying to compile reports:

```
assertThrows(ResponseStatusException.class, () -> {  
    reportService.generateReport(requestDTO);  
});
```